

Automat na kuličky

Medvěd vytvořil automat na kuličky, který má pobavit účastníky IOI. Vnitřní struktura automatu je následující:

- Automat se skládá z N **vrcholů**, indexovaných od 0 do $N - 1$. Vrchol $N - 1$ se nazývá **kořen**.
- Pro každý vrchol u ($0 \leq u < N - 1$) platí, že **rodičem** vrcholu u je vrchol $P[u]$ ($P[u] > u$) a **potomkem** vrcholu $P[u]$ je vrchol u . Kořen nemá žádného rodiče. Vrchol bez potomků se nazývá **list**.

Nevíte, kolik je N , ani jaký je rodič $P[u]$ každého vrcholu u . Místo toho vám Medvěd sdělil, kolik je M (počet listů) a že indexy listů jsou $0, 1, \dots, M - 1$. Vaším úkolem je určit vnitřní strukturu automatu tím, že jej budete ovládat.

Každý vrchol automatu může obsahovat nanejvýš jednu **kuličku** a každá kulička má **hodnotu**, která je nezáporným celým číslem. Řekneme, že vrchol má hodnotu x , pokud obsahuje kuličku s hodnotou x . Vrchol je **obsazený**, pokud obsahuje kuličku; v opačném případě je **volný**. Zpočátku je každý vrchol volný.

Můžete provádět dva druhy operací:

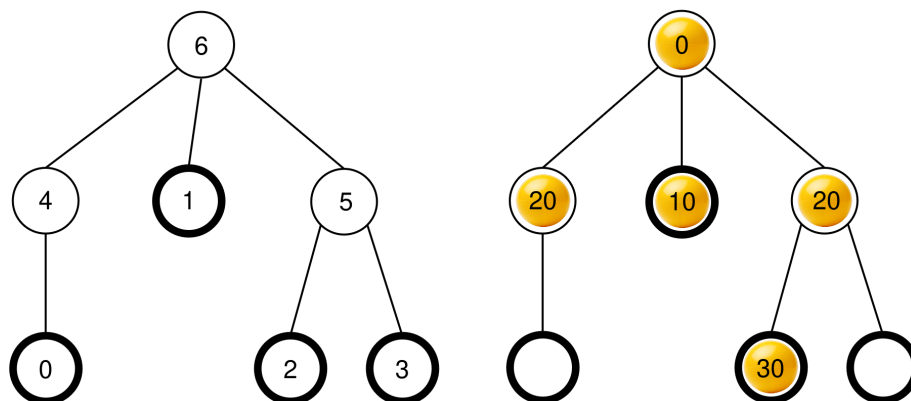
- **vložení** (U, X): pokusí se vložit novou kuličku s hodnotou X do listu U .
 - Pokud je list U obsazen, operace vrátí `false` bez vložení kuličky.
 - Pokud je list U volný, kulička se umístí do vrcholu U . Poté se kulička opakovaně přesouvá do rodiče aktuálního vrcholu, dokud je tento rodič volný. Přesun se zastaví, jakmile kulička dosáhne kořene nebo vrcholu, jehož rodič je obsazen. Operace vrátí hodnotu `true`.

- **collect():** pokud je kořen volný (což znamená, že automat je prázdný), vrátí `collect` prázdné pole. V opačném případě rekurzivní funkce `traverse` popsaná níže shromáždí hodnoty všech kuliček, které se aktuálně nacházejí v automatu, do pole S . Zpočátku je pole S prázdné a je volána funkce `traverse(N - 1)`.

```
traverse(u):
    přidej hodnotu kuličky ve vrcholu u na konec S
    nechť c[u] je seznam obsazených potomků u
    setříd' c[u] do neklesajícího pořadí dle hodnot kuliček
    ve vrcholech (pokud má více stejnou hodnotu, mohou se
    objevit v libovolném pořadí)
    pro každý v z c[u]:
        traverse(v)
```

Po dokončení této funkce jsou **všechny kuličky odstraněny** z automatu a funkce `collect` vrátí S .

Uvažujme například automat s $N = 7$ vrcholy a polem rodičů $P = [4, 6, 5, 5, 6, 6]$. Obrázek vlevo zobrazuje prázdný automat s indexy vrcholů, zatímco obrázek vpravo ukazuje možnou konfiguraci kuliček po určité sekvenci operací `insert` (tato sekvence je uvedena v sekci Příklad).



Při volání funkce `collect()` je kořen (vrchol 6) obsazen, takže proměnná S je inicializována jako $S = []$ a je volána funkce `traverse(6)`.

traverse(6): vrchol 6 má hodnotu 0; jejím připojením k S dostaneme $S = [0]$. Děti vrcholu 6 jsou 4, 1 a 5, které jsou všechny obsazené. Jejich hodnoty jsou 20, 10 a 20. Řazení v neklesajícím pořadí dává $c[6] = [1, 5, 4]$ (podotkněme, že $c[6] = [1, 4, 5]$ by bylo také platné, jelikož vrcholy 4 a 5 mají stejnou hodnotu). Funkce poté zavolá `traverse` na každém vrcholu v $c[6]$ v tomto pořadí.

- **traverse(1)**: vrchol 1 má hodnotu 10; jejím připojením k S získáme $S = [0, 10]$. Vrchol 1 nemá žádné obsazené potomky, takže toto volání končí.
- **traverse(5)**: vrchol 5 má hodnotu 20; jejím připojením k S dostaneme $S = [0, 10, 20]$. Vrchol 5 má jednoho obsazeného potomka, vrchol 2, takže $c[5] = [2]$ a funkce volá `traverse(2)`.
 - **traverse(2)**: vrchol 2 má hodnotu 30; jejím připojením k S dostaneme $S = [0, 10, 20, 30]$. Vrchol 2 nemá žádné obsazené potomky, takže toto volání končí.
 - Provedení se vrátí do `traverse(5)`, která zpracovala všechny potomky v $c[5]$, takže její volání končí.
- **traverse(4)**: vrchol 4 má hodnotu 20; jejím připojením k S dostaneme $S = [0, 10, 20, 30, 20]$. Vrchol 4 nemá žádné obsazené potomky, takže toto volání končí.

Všechna volání byla dokončena a již nejsou žádné další vrcholy ke zpracování. Nakonec jsou všechny kuličky vyjmuty z automatu a funkce `collect` vrátí pole $S = [0, 10, 20, 30, 20]$.

Vaším úkolem je určit počet vrcholů N a nahlásit pole rodičů $R = [R[0], R[1], \dots, R[N-2]]$, které popisuje strukturu automatu, avšak s potenciálně odlišným označením těch vrcholů, které nejsou listy, ani kořenem. Formálně je odevzdané pole R považováno za správné, pokud je možné každému vrcholu u automatu přiřadit jedinečný label $L[u]$, kde $0 \leq L[u] < N$, tak, že:

- $L[u] = u$ pro všechna $0 \leq u < M$ a $u = N - 1$ a
- $L[P[u]] = R[L[u]]$ pro všechna $0 \leq u < N - 1$.

Není nutné, aby platilo $R[u] > u$. Automat **musí být prázdný**, když končí `find_structure`.

Nechť K je počet provedených operací `collect` a B je maximální hodnota kuliček ze všech volání `insert`. Nechť $C = K + B$. Pak C **nesmí překročit** 1000. Vaše bodování v některých podúlohách závisí na hodnotě C .

Implementační detaily

Měli byste implementovat následující funkci.

```
std::vector<int> find_structure(int M)
```

- M : počet listů v automatu.
- Funkce je volána právě jednou v každém testu.
- Funkce musí vrátit pole $R = [R[0], R[1], \dots, R[N - 2]]$ o délce $N - 1$, správné pole rodičů.

Pro interakci s automatem může vaše funkce volat následující dvě funkce.

První funkce je:

```
bool insert(int U, int X)
```

- U : index listu, do kterého má být kulička vložena. Musí platit $0 \leq U < M$.
- X : celočíselná hodnota kuličky. Musí být splněna podmínka $0 \leq X \leq 1000$.
 - Tato funkce vrací hodnotu `true`, pokud byla kulička úspěšně vložena, nebo hodnotu `false`, pokud byl list U již obsazen.
- Tato funkce může být volána maximálně 500 000 krát.

Druhá funkce je:

```
std::vector<int> collect()
```

- Shromáždí hodnoty všech vložených kuliček, vyprázdní automat a vrátí shromážděné pole S .

Pokud v jakémkoli okamžiku během provádění vašeho programu hodnota C překročí 1000, vaše řešení obdrží verdikt `Output isn't correct: Too many resources used`.

Struktura automatu je pevně daná před voláním `find_structure`. Grader je **deterministický** v tom smyslu, že pokud je puštěn dvakrát a obě puštění vykonají stejnou posloupnost operací, volání funkce `collect` vrátí stejná pole.

Omezení

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Podúlohy

Podúloha	Body	Omezení
1	5	Kořen má přesně M potomků.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Žádná další omezení.

V podúlohách 3 a 4 závisí vaše skóre na hodnotě C takto:

Podúloha 3 (25 bodů)

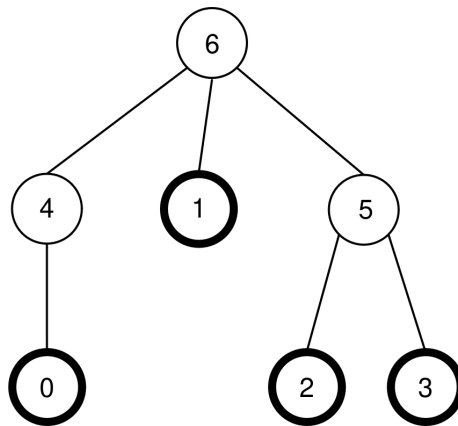
Podmínka	Bodování
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Podúloha 4 (60 bodů)

Podmínka	Bodování
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Příklad

Uvažujme ten samý automat jako v popisu úlohy, tedy $N = 7$, $M = 4$ a $P = [4, 6, 5, 5, 6, 6]$. Automat je opět znázorněn na následujícím obrázku:



Grader provede následující volání:

```
find_structure(4)
```

Funkce může provést následující posloupnost volání:

1. **insert(0, 0):** list 0 je volný, takže kulička s hodnotou 0 je umístěna na list 0. Vrchol $P[0] = 4$ je volný, takže kulička se přesune do vrcholu 4. Vrchol $P[4] = 6$ je volný, takže kulička se přesune do vrcholu 6. Protože vrchol 6 je kořenem, pohyb se zastaví. Volání vrátí `true`.
2. **insert(3, 20):** list 3 je volný, takže kulička s hodnotou 20 je umístěna na list 3. Vrchol $P[3] = 5$ je volný, takže kulička se přesune do vrcholu 5. Protože je $P[5] = 6$ obsazený, pohyb se zastaví. Volání vrátí `true`.
3. **insert(1, 10):** list 1 je volný, takže na list 1 umístíme kuličku s hodnotou 10. Protože $P[1] = 6$ je obsazené, kulička zůstává na vrcholu 1. Volání vrátí `true`.
4. **insert(2, 30):** list 2 je volný, takže na list 2 umístíme kuličku s hodnotou 30. Protože $P[2] = 5$ je obsazené, kulička zůstává na vrcholu 2. Volání vrátí `true`.
5. **insert(1, 25):** list 1 je obsazený, takže kulička není umístěna na list 1. Volání vrátí `false`.
6. **insert(0, 20):** list 0 je volný, takže kulička s hodnotou 20 je umístěna na list 0. Vrchol $P[0] = 4$ je volný, takže kulička se přesune do vrcholu 4. Protože $P[4] = 6$ je obsazený, pohyb se zastaví. Volání vrátí `true`.

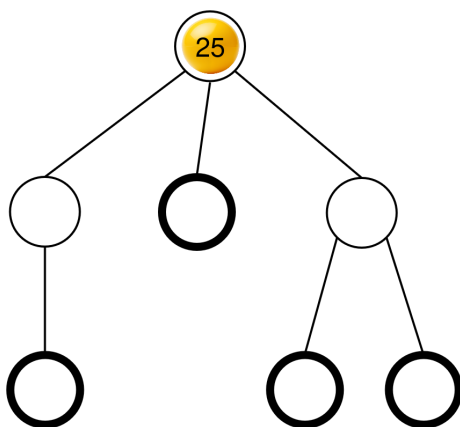
Výsledná konfigurace kuliček byla již zobrazena v popisu úlohy.

Dále může funkce volat:

```
collect()
```

Provedení tohoto volání již bylo vysvětleno v popisu úlohy, takže toto volání vrátí $S = [0, 10, 20, 30, 20]$. Poté je automat opět prázdný.

Dále může funkce zavolat `insert(2, 25)`. List 2 je volný, takže kulička s hodnotou 25 je umístěna na list 0. Tato kulička se poté přesune do vrcholu 5, a poté do vrcholu 6, kde se zastaví. Volání vrátí `true`. Výsledná konfigurace kuliček je znázorněna na následujícím obrázku.



Nakonec může funkce zavolat funkci `collect()`, která vrátí $S = [25]$ a vyprázdní automat.

Funkce `find_structure` může vrátit dvě možná pole rodičů: $R = P = [4, 6, 5, 5, 6, 6]$ (což odpovídá označení $L = [0, 1, 2, 3, 4, 5, 6]$) a $R = [5, 6, 4, 4, 6, 6]$ (což odpovídá označení $L = [0, 1, 2, 3, 5, 4, 6]$). V tomto příkladu je $K = 2$ a $B = 30$, takže $C = 32$.

Ukázkový grader

Vstupní formát:

```
NM
P[0] P[1] P[2] ... P[N-2]
```

Výstupní formát:

```
QR[0] R[1] R[2] ... R[Q-1]
```

Kde Q je délka pole R vráceného funkcí `find_structure`.